

Competitive Programming

Section 27: Bitmask DP

Competitive Programming Course

2026-08-06

Table of contents

1	Why bitmask DP matters	3
2	Vocabulary and bit operations	4
3	Recognizing bitmask DP	5
4	Pattern 1: coverage DP	6
4.1	Begin with an explicit candidate dimension	6
4.2	Remove the candidate dimension with two layers	7
4.3	Trace the masks, not only the formula	7
5	Pattern 2: assignment DP	9
6	Pattern 3: subset plus last	10
7	Iteration order	11
8	Complexity intuition	13
9	C++ implementation notes	14
10	Python implementation notes	15
11	Proof habits	16
12	Common mistakes	17
13	Local judge workflow	18
14	Submission checklist	19
15	Exercises	20

1 Why bitmask DP matters

Many contest problems ask you to make decisions about a small set:

```
Which skills are already covered?  
Which jobs have already been used?  
Which cities have already been visited?  
Which special vertices have already been collected?
```

If the number of items is large, a set state is impossible. But when the statement says $n \leq 16$, $n \leq 18$, or $n \leq 20$, the intended solution is often to store the set as an integer.

This is **bitmask DP**: dynamic programming where a bit pattern represents a subset.

The key contest skill is not memorizing a formula. It is deciding:

```
What does this mask mean?  
What extra information, if any, must be stored besides the mask?  
How do I add one item while preserving the meaning of the state?
```

2 Vocabulary and bit operations

For n items, a mask is an integer from 0 to $2^n - 1$.

Bit i is 1 if item i is present and 0 otherwise.

For four items:

```
mask = 13
binary = 1101

item:  3 2 1 0
bit:   1 1 0 1
set:   {0, 2, 3}
```

Common C++ operations:

Operation	C++	Meaning
full mask	$(1 \ll n) - 1$	all items present
test bit i	$(\text{mask} \gg i) \& 1$	is item i present?
remove item i	$\text{mask} \& \sim(1 \ll i)$	force bit i to 0
intersection	$a \& b$	items in both sets
count bits	<code>__builtin_popcount(mask)</code>	subset size

The two common bitwise-OR operations are clearer outside a Markdown table, where the pipe character also separates columns:

```
mask | (1 << i) // Add item i.
a | b          // Union: items in either set.
```

The Python expressions are the same except for counting set bits:

```
mask.bit_count()
```

In prose, we usually number items from 0 internally even if input is 1-indexed.

3 Recognizing bitmask DP

Bitmask DP is plausible when:

Constraint or shape	Typical interpretation
$n \leq 20$	one-dimensional DP over all subsets
$n \leq 18$ and assignment/permutation	<code>dp[mask]</code> , next row from <code>popcount(mask)</code>
$n \leq 16$ and route/order matters	<code>dp[mask][last]</code>
all subsets of a small set appear	enumerate masks from 0 to $2^n - 1$
every object is used once	mask marks used objects
shortest route visits special nodes	mask marks visited special nodes, plus current node

It is often wrong when $n = 10^5$. A 2^n table is only for small n .

4 Pattern 1: coverage DP

Suppose we need three skills, numbered 0, 1, and 2. We may pay for candidates, and every candidate teaches a particular subset of those skills. The goal is to obtain all three skills as cheaply as possible.

The mask describes **skills already covered**, not candidates already chosen:

```
000 = no skill is covered
001 = only skill 0 is covered
011 = skills 0 and 1 are covered
111 = every skill is covered
```

Several different candidate selections may produce the same mask. Once they cover the same skills, the future does not care which candidates produced them; it only cares which selection was cheaper. This is the compression that makes the DP possible.

4.1 Begin with an explicit candidate dimension

A completely explicit state would be:

```
dp[i][mask] = minimum cost of obtaining exactly mask
               using only the first i candidates
```

When processing candidate *i*, there are only two final choices:

```
skip the candidate
take the candidate
```

If the candidate costs *c* and covers *offer*, skipping keeps the old mask and cost. Taking it combines the skills already covered with the new skills:

```
new_mask = mask | offer
```

For example:

```
already covered: 011
new offer:       101
union:           111
```

Skill 0 appearing in both masks is still just one covered skill. Bitwise OR expresses exactly this union.

4.2 Remove the candidate dimension with two layers

Only the preceding candidate layer is needed. Let `dp[mask]` describe the old layer and build `next_dp` for the new layer:

A mask is **reachable** when `dp[mask]` is not `INF`: at least one selection of the candidates processed so far produces exactly that set of covered skills. Masks whose value is `INF` do not represent any real selection and must not create transitions.

```
next_dp = dp

for every mask whose dp[mask] is not INF:
    new_mask = mask | offer
    next_dp[new_mask] = min(next_dp[new_mask], dp[mask] + cost)

dp = next_dp
```

Copying `dp` first represents skipping the candidate. Every update that adds its cost represents taking it. Reading transitions only from the old `dp` layer ensures that the current candidate is used at most once.

4.3 Trace the masks, not only the formula

Consider:

```
Candidate A: cost 5, covers 011
Candidate B: cost 4, covers 100
Candidate C: cost 7, covers 101
```

Initially, choosing nobody is the only possibility:

```
dp[000] = 0
all other masks are unreachable
```

After A, we may skip it or take it:

```
dp[000] = 0    # skip A
dp[011] = 5    # take A
```

When B is processed, each existing state again branches:

```
from 000: skip B -> 000 with cost 0
from 000: take B -> 100 with cost 4
from 011: skip B -> 011 with cost 5
from 011: take B -> 111 with cost 9
```

Candidate C alone reaches 101 for cost 7, but it does not produce full coverage more cheaply than A plus B. Therefore `dp[111]` remains 9.

The invariant after each layer is:

For every mask, `dp[mask]` is the cheapest selection among the candidates processed so far whose union of covered skills is exactly that mask.

The answer is the entry for the full mask `(1 << skill_count) - 1`.

5 Pattern 2: assignment DP

There are n workers and n jobs. Each worker must get one unused job.

A direct state might seem to need both worker index and job set:

```
dp[i][mask] = best after assigning first i workers and using jobs in mask
```

But i is already determined by the mask:

```
i = number of used jobs = popcount(mask)
```

So the state can be one-dimensional:

```
dp[mask] = best profit after assigning the first popcount(mask) workers  
           to exactly the jobs in mask
```

Transition:

```
i = popcount(mask)  
for each unused job j:  
    next = mask | (1 << j)  
    dp[next] = max(dp[next], dp[mask] + profit[i][j])
```

Small trace with two workers:

```
profit =  
worker 0: job0=3 job1=8  
worker 1: job0=6 job1=4
```

States:

```
dp[00] = 0  
  
mask 00, worker 0:  
    choose job 0 -> dp[01] = 3  
    choose job 1 -> dp[10] = 8  
  
mask 01, worker 1:  
    choose job 1 -> dp[11] = 7  
  
mask 10, worker 1:  
    choose job 0 -> dp[11] = max(7, 14)
```

Answer: 14.

6 Pattern 3: subset plus last

If the order matters, the mask alone is not enough.

For a route problem:

```
dp[mask] = best cost to have visited mask
```

does not tell us where we are standing, so we do not know the cost of the next move. We need:

```
dp[mask][last] = minimum cost to start at 0,
                  visit exactly the vertices in mask,
                  and finish at last
```

Initial state:

```
start_mask = 1 << 0
dp[start_mask][0] = 0
```

The route starts at vertex 0, so vertex 0 has already been visited and bit 0 must be 1. No edge has been travelled yet, so the cost is 0. In binary, `start_mask` ends in 1:

```
...0001
```

Starting from mask 0 would contradict the state definition: it would say that the route finishes at vertex 0 while vertex 0 is not in the visited set.

Transition:

```
for each unvisited nxt:
    dp[mask | (1 << nxt)][nxt]
        = min(dp[...], dp[mask][last] + cost[last][nxt])
```

Final answer for a cycle returning to start:

```
min over last: dp[full][last] + cost[last][0]
```

Diagram:

```
state: mask = {0, 2, 4}, last = 2
```

```
visited:  0 ---- 4
           \    |
            \    |
             2  <- current location
```

```
unvisited choices can be appended one at a time
```

7 Iteration order

Most beginner bitmask DP tables can be filled by increasing mask:

```
for (int mask = 0; mask < (1 << n); mask++) {  
    ...  
}
```

This can feel unlike earlier DP orders. A larger integer does **not** necessarily contain more selected items:

```
0111 = 7 has three set bits  
1001 = 9 has two set bits
```

Thus increasing numeric order is not the same as processing layers with one visited city, then two, then three.

The reason numeric order works for Inspection Route is narrower: every transition **strictly adds at least one previously zero bit** and never removes one. If **next** is not yet in **mask**, then

```
new_mask = mask | (1 << next)
```

sets one previously zero bit. Consequently **new_mask > mask**. More generally, if mask A is a proper subset of mask B, then the integer encoding of A is smaller than the encoding of B.

For Inspection Route, every incoming transition to **(mask, last)** comes from a mask with one visited city removed. That predecessor is therefore numerically smaller and has already been processed. A later numeric mask with fewer set bits, such as 1001 after 0111, is unrelated to 0111; neither is a subset of the other, so no dependency is being processed backward.

For **dp[mask][last]**, use the same outer mask loop, then loop over **last**.

You may instead group masks by **mask.bit_count()** and process groups from small to large. That order mirrors the number of visited cities and may be easier to reason about, but it does not reduce the total number of masks. The simple increasing-mask loop is usually preferred once the subset-order argument is clear.

Merely saying that a transition “does not remove bits” is not sufficient. It might leave the mask unchanged:

```
dp[mask][a] -> dp[mask][b]
```

The outer mask loop gives no ordering between those two states; the inner dimension would need its own valid evaluation order. Likewise, **new_mask = mask | offer** may equal **mask** when every offered bit is already present. In coverage DP that same-mask transition cannot improve the answer because it only adds a nonnegative cost, but that fact needs a separate proof.

The precise checklist is:

Every dependency from a different mask comes from a proper submask.
Every dependency that keeps the same mask is harmless or is processed
in a separately justified inner order.

If a transition removes a bit, or replaces one bit with another, increasing numeric order needs a new justification and may be wrong.

8 Complexity intuition

There are 2^n masks.

DP shape	States	Transition factor	Typical complexity
coverage	2^n	candidates	$O(m \cdot 2^n)$
assignment	2^n	unused jobs	$O(n \cdot 2^n)$
route/TSP	$n \cdot 2^n$	next vertex	$O(n^2 \cdot 2^n)$

Rough limits in C++:

```
2^20 ~= 1,048,576      usually fine
20 * 2^20 ~= 21 million
20^2 * 2^20 ~= 419 million, often too high
16^2 * 2^16 ~= 16 million, fine
```

Python is slower, but these local problems keep sizes within reasonable bounds for clean Python.

9 C++ implementation notes

Use `long long` for costs and profits.

```
const long long INF = (1LL << 62);  
vector<long long> dp(1 << n, INF);
```

For bit counts:

```
int k = __builtin_popcount((unsigned)mask);
```

When `n` can be 20 or more, write `1 << n` only if `n` fits in 31 bits. In this course section, `n` $\leq$ 20, so `int` masks are fine.

10 Python implementation notes

Python has arbitrary precision integers, so bit shifts are convenient:

```
full = (1 << n) - 1
if (mask >> i) & 1:
    ...
```

Use `mask.bit_count()` for popcount. It is implemented efficiently.

For large DP lists, avoid dictionaries unless the reachable state set is truly sparse. Lists are faster:

```
dp = [10**30] * (1 << n)
```

11 Proof habits

Bitmask DP proofs usually follow induction on subset size.

Template:

```
Base: the empty or start mask has the correct value.  
Induction step: assume all states with k chosen items are correct.  
Every state with k+1 chosen items is reached by adding one valid item to a  
correct k-item state, and the transition takes the best among all such last  
choices.  
Therefore the new state is correct.
```

For route DP, the proof must mention why `last` is enough. Once the visited set and current vertex are known, the future cost does not depend on the exact previous order except through the accumulated optimal cost stored in the DP.

12 Common mistakes

```
Using input skill numbers directly without subtracting 1.  
Forgetting that full mask is  $(1 \ll n) - 1$ , not  $1 \ll n$ .  
Using dp[mask] for route problems where current position matters.  
Using popcount(mask) after adding the next job instead of before.  
Returning dp[full] in TSP without adding the edge back to the start.  
Allocating  $O(n \cdot 2^n)$  memory when  $O(2^n)$  is enough.  
Using int for answers that can exceed  $2^{31} - 1$ .
```

13 Local judge workflow

Run only this section:

```
CP_TARGET=solution python3 sections/27_bitmask_dp/check.py
```

When solving yourself, omit `CP_TARGET=solution`:

```
python3 sections/27_bitmask_dp/check.py
```

The randomized tests include edge cases such as one item, asymmetric route costs, and impossible coverage.

14 Submission checklist

Before submitting a bitmask DP solution:

1. Write down what each bit means.
2. Write down what `dp[mask]` or `dp[mask][last]` means.
3. Confirm the start state.
4. Confirm transitions only use valid missing/present bits.
5. Confirm the answer state: full mask, maybe plus final return edge.
6. Estimate states times transition factor.
7. Use long long / Python int for large sums.

15 Exercises

Use *A* to make the covered-skill mask explicit, *B* to match the number of assigned workers to the mask popcount, and *C* to state the start and end of the route. Test mask zero, the full mask, and one missing transition.